COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:		**DATE SUBMITTED:**	
NAME:	Adrian Walter D. Santos	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO.3 **Linked lists**

1. PROCEDURE OUTPUT

Screenshot

```
#include <iostream>
#include "singly_ll.h"

class Node{
    public:
        char data;
        Node *next;
};

int main(){
    Node* head = nullptr;
    Node* second = nullptr;
    Node* third = nullptr;
    Node* fourth = nullptr;
    Node* fifth = nullptr;

    head = new Node;
    second = new Node;
    third = new Node;
    fourth = new Node;
    fifth = new Node;
```

```
23      head->data = 'J';
24      head->next = second;
25
26      second->data = 'I';
27      second->next = third;
28
29      third->data = 'M';
30      third->next = fourth;
31
32      fourth->data = 'Q';
33      fourth->next = fifth;
34
35      fifth->data = 'U';
36      fifth->next = nullptr;
37
38      delete head;
39      delete second;
40      delete third;
41      delete fourth;
42      delete fifth;
43
44
45      return 0;
46  }
```

Discussion	This part showed us how to create our own linked lists. Which instructed us to first create the node pointers then create new nodes allocated in the heap, then we defined every node in the list and finally pointed the last node to null. This gave us the basic knowledge and steps in order to make our own linked lists so now we are able to utilize linked lists in our future coding endeavors..

Operation	Screenshot
Traversal	

```cpp
#ifndef SINGLY_LL_H
#define SINGLY_LL_H

template <typename T>
class SingleList{
    public:
    T data;
    SingleList<T> *next = nullptr;
};

template <typename T>
void listTraversal(SingleList<T>* head){
    while( head != nullptr){
        std::cout << head->data;

        //To check if the next linew is not equal to null
        if (head->next != nullptr){
            std::cout << "->";
        }
        head = head->next;
    }
    std::cout << std::endl;
}

#endif
```

```cpp
#include <iostream>
#include "singly_ll.h"



int main(){
    SingleList<int> *head = new SingleList<int>;
    SingleList<int> *second = new SingleList<int>;
    SingleList<int> *third = new SingleList<int>;

    head->data = 1;
    head->next = second;

    second->data = 2;
    second->next = third;

    third->data = 3;
    third->next = nullptr;

    std::cout << "Linked list traversal: ";
    listTraversal(head);

    delete head;
    delete second;
    delete third;

    return 0;
}
```

Insertion at the head

```
template <typename T>
void sllinsertHead(T newData, SingleList <T>** currentHead){

    //
    SingleList<T>* newNode = new SingleList<T>;
    //putour data into the new node
    newNode->data = newData;
    //set next of the new  nod to the point of the previous head
    newNode->next = *currentHead;
    //reset Head to point to the new node
    *currentHead = newNode;
}
```

Insertion at any part

```
template <typename T>
void sllGeneralinsertion(T newData, SingleList<T> * prevNode){
    if(prevNode==nullptr){
        std::cout<<"Previous node cannot be null. "<<std::endl;
        return;
    }
    //Allocate a new node
    SingleList<T> * newNode = new SingleList<T>;
    //store datra inew new node
    newNode->data = newData;
    //point new node to the node previous node was pointing to
    newNode->next = prevNode->next;
    // point previous node to the new node
    prevNode->next = newNode;

}
#endif
```

Insert at the end

```
void sllinsertEnd(T newData, SingleList<T>** head){
    //allocate new node
    SingleList<T>* newNode = new SingleList<T>;
    //dereference to the head node
    SingleList<T>* currentNode = *head;
    //store data in new node
    newNode->data = newData;
    //poinr nextr of new node to null
    newNode->next = nullptr;

    //check if list is empty
    if(currentNode == nullptr){
        currentNode = newNode;
        return;
    }
    //traverse the list until nextof the node is null
    while(currentNode->next != nullptr){
        //iteration point to one node to another
        currentNode = currentNode->next;
    }
    currentNode -> next = newNode;
}
```

Delete of a node

```
template <typename T>
void sllDelete(T findData, SingleList<T>** head){
    //check if there is a list
    if(*head == nullptr) return;

    SingleList<T>* currNode = *head;
    SingleList<T>* prevNode = nullptr;

    //search for the node
    while(currNode != nullptr && currNode->data != findData){
        prevNode = currNode;
        currNode = currNode->next;

    }
    //if data is not found
    if(currNode == nullptr) return;

    //delete the head node (the head node is the findData)
    if (prevNode ==nullptr){
        *head = currNode->next;
    }else{
        prevNode->next = currNode->next;
    }

    delete currNode;
}
```

	Source Code	

```cpp
#include <iostream>
#include "singly_ll.h"

int main() {
    SingleList<char> *head = new SingleList<char>;
    SingleList<char> *second = new SingleList<char>;
    SingleList<char> *third = new SingleList<char>;
    SingleList<char> *fourth = new SingleList<char>;
    SingleList<char> *fifth = new SingleList<char>;
    SingleList<char> *sixth = new SingleList<char>;

    //creating the data and the link nodes
    head->data = 'C';
    head->next = second;

    second->data = 'P';
    second->next = third;

    third->data = 'E';
    third->next = fourth;

    fourth->data = '0';
    fourth->next = fifth;

    fifth->data = '1';
    fifth->next = sixth;

    sixth->data = '0';
    sixth->next = nullptr;
```

A.	**Source Code**	

```cpp
listTraversal(head);
```

	Console	

```
C->P->E->0->1->0
```

B.	**Source Code**	

```cpp
sllinsertHead('G', &head);
std::cout << "After inserting 'G' at the head: ";
listTraversal(head);
```

	Console	

```
After inserting 'G' at the head: G->C->P->E->0->1->0
```

C.	**Source Code**	`sllGeneralinsertion('E', head->next->next);`
		`std::cout << "After general insertion of 'E': ";`
		`listTraversal(head);`

	Console	`After general insertion of 'E': G->C->P->E->E->0->1->0`

D.	**Source Code**	`sllDelete('C', &head);`
		`std::cout << "After deleting 'C': ";`
		`listTraversal(head);`

	Console	`After deleting 'C': G->P->E->E->0->1->0`

E.	**Source Code**	`sllDelete('P', &head);`
		`std::cout << "After deleting 'P': ";`
		`listTraversal(head);`

	Console	`After deleting 'P': G->E->E->0->1->0`

F. **Source Code**

```cpp
#include <iostream>
#include "singly_ll.h"

int main() {
    SingleList<char> *head = new SingleList<char>;
    SingleList<char> *second = new SingleList<char>;
    SingleList<char> *third = new SingleList<char>;
    SingleList<char> *fourth = new SingleList<char>;
    SingleList<char> *fifth = new SingleList<char>;
    SingleList<char> *sixth = new SingleList<char>;

    //creating the data and the link nodes
    head->data = 'C';
    head->next = second;

    second->data = 'P';
    second->next = third;

    third->data = 'E';
    third->next = fourth;

    fourth->data = '0';
    fourth->next = fifth;

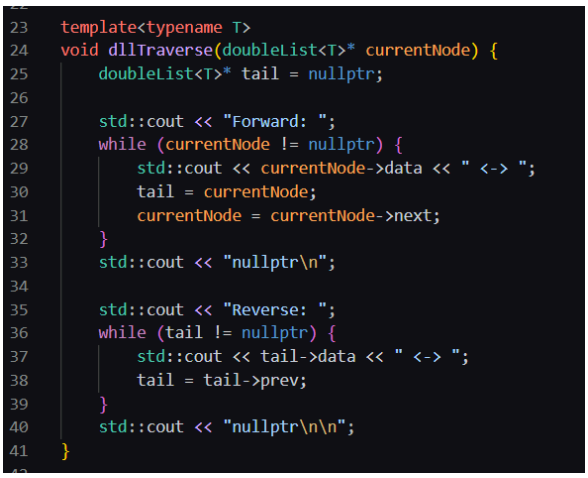
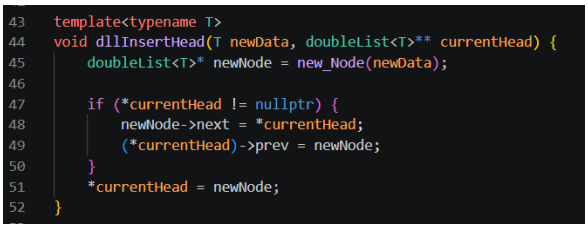
    fifth->data = '1';
    fifth->next = sixth;

    sixth->data = '0';
    sixth->next = nullptr;
```

```cpp
    listTraversal(head);
    sllinsertHead('G', &head);
    std::cout << "After inserting 'G' at the head: ";
    listTraversal(head);
    sllGeneralinsertion('E', head->next->next);
    std::cout << "After general insertion of 'E': ";
    listTraversal(head);
    sllDelete('C', &head);
    std::cout << "After deleting 'C': ";
    listTraversal(head);
    sllDelete('P', &head);
    std::cout << "After deleting 'P': ";
    listTraversal(head);
    return 0;
}
```

	Console	

```
C->P->E->0->1->0
After inserting 'G' at the head: G->C->P->E->0->1->0
After general insertion of 'E': G->C->P->E->E->0->1->0
After deleting 'C': G->P->E->E->0->1->0
After deleting 'P': G->E->E->0->1->0
```

Screenshot	Analysis
	This function prints out all the items in a doubly linked list, going forward first and then backward. As it loops forward through each node to print its contents, it saves the location of the last node in a temporary tail variable. It then uses that saved tail node to step backward through the list, printing every item in reverse order until it reaches the start.
	This function adds a new item to the very start of a doubly linked list. It creates the new node first and, if the list isn't empty, connects it to the old first node in both directions. Finally, it updates the head pointer so the new node becomes the starting node of the list.
	This function adds a new item to the very end of a doubly linked list by creating a node first. If the list is completely empty, it sets the new node as the start of the list and finishes right away. Otherwise, it walks all the way to the last node and attaches the new node to the end, connecting their forward and backward links.

```
template<typename T>
void dllTraverse(doubleList<T>* currentNode) {
    doubleList<T>* tail = nullptr;

    std::cout << "Forward: ";
    while (currentNode != nullptr) {
        std::cout << currentNode->data << " <-> ";
        tail = currentNode;
        currentNode = currentNode->next;
    }
    std::cout << "nullptr\n";

    std::cout << "Reverse: ";
    while (tail != nullptr) {
        std::cout << tail->data << " <-> ";
        tail = tail->prev;
    }
    std::cout << "nullptr\n\n";
}
```

```
template<typename T>
void dllInsertHead(T newData, doubleList<T>** currentHead) {
    doubleList<T>* newNode = new_Node(newData);

    if (*currentHead != nullptr) {
        newNode->next = *currentHead;
        (*currentHead)->prev = newNode;
    }
    *currentHead = newNode;
}
```

```
template<typename T>
void dllInsertTail(T newData, doubleList<T>** currentHead) {
    doubleList<T>* newNode = new_Node(newData);

    if (*currentHead == nullptr) {
        *currentHead = newNode;
        return;
    }

    doubleList<T>* temp = *currentHead;
    while (temp->next != nullptr) {
        temp = temp->next;
    }
    temp->next = newNode;
    newNode->prev = temp;
}
```



```
template<typename T>
bool dllInsert(T newData, int index, doubleList<T>** currentHead) {
    if (index < 0) return false;

    // Insert at head
    if (index == 0) {
        dllInsertHead(newData, currentHead);
        return true;
    }

    doubleList<T>* temp = *currentHead;
    for (int i = 0; temp != nullptr && i < index - 1; ++i) {
        temp = temp->next;
    }

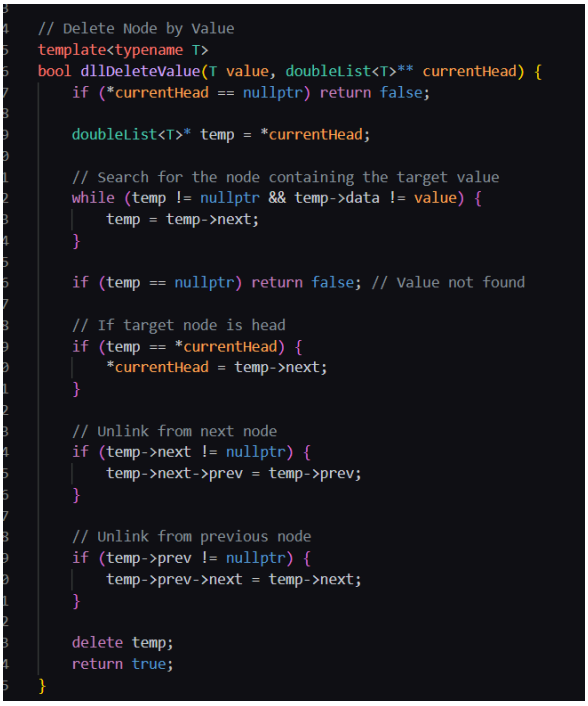
    // Index out of bounds
    if (temp == nullptr) return false;

    doubleList<T>* newNode = new_Node(newData);
    newNode->next = temp->next;
    newNode->prev = temp;

    if (temp->next != nullptr) {
        temp->next->prev = newNode;
    }
    temp->next = newNode;

    return true;
}
```

This function adds a new item at a specific position in a doubly linked list by counting forward to the spot right before it. It returns false if the position is negative or past the end of the list, but uses a helper function to easily handle inserting at the very start. Once it finds the right spot, it creates a new node, hooks up the surrounding links to include it, and returns true.



```
// Delete Node by Value
template<typename T>
bool dllDeleteValue(T value, doubleList<T>** currentHead) {
    if (*currentHead == nullptr) return false;

    doubleList<T>* temp = *currentHead;

    // Search for the node containing the target value
    while (temp != nullptr && temp->data != value) {
        temp = temp->next;
    }

    if (temp == nullptr) return false; // Value not found

    // If target node is head
    if (temp == *currentHead) {
        *currentHead = temp->next;
    }

    // Unlink from next node
    if (temp->next != nullptr) {
        temp->next->prev = temp->prev;
    }

    // Unlink from previous node
    if (temp->prev != nullptr) {
        temp->prev->next = temp->next;
    }

    delete temp;
    return true;
}
```

This function removes a matching item from a doubly linked list by walking through the nodes starting at the head. It handles special cases safely, such as empty lists, missing values, or deleting the first node in line. Once it detaches the matching node from its neighbors, it frees up that node's memory and returns true.

```cpp
// Delete Node by 0-based Index
template<typename T>
bool dllDeleteIndex(int index, doubleList<T>** currentHead) {
    if (*currentHead == nullptr || index < 0) return false;

    doubleList<T>* temp = *currentHead;

    for (int i = 0; temp != nullptr && i < index; ++i) {
        temp = temp->next;
    }

    if (temp == nullptr) return false; // Index out of bounds

    if (temp == *currentHead) {
        *currentHead = temp->next;
    }

    if (temp->next != nullptr) {
        temp->next->prev = temp->prev;
    }

    if (temp->prev != nullptr) {
        temp->prev->next = temp->next;
    }

    delete temp;
    return true;
}
```

This function removes an item at a specific position in a doubly linked list by counting forward from the start. It safely stops and returns false if the list is empty, if the given position is negative, or if it goes past the end. Once it finds the right node, it updates the surrounding links, deletes the node to clean up memory, and returns true.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

CODE

```cpp
#ifndef SINGLY_LL_H
#define SINGLY_LL_H

#include <iostream>

template <typename T>
class SingleList {
public:
    T data;
    SingleList<T>* next = nullptr;
};

template <typename T>
void listTraversal(const SingleList<T>* head) {
    if (head == nullptr) {
        std::cout << "Playlist is empty.\n";
        return;
    }
    const SingleList<T>* temp = head;
    int i = 0;
    do {
        std::cout << ++i << ". " << temp->data << "\n";
        temp = temp->next;
    } while (temp != head);
}
```

```cpp
template <typename T>
void sllinsertEnd(const T& newData, SingleList<T>*& head) {
    SingleList<T>* newNode = new SingleList<T>;
    newNode->data = newData;

    if (head == nullptr) {
        head = newNode;
        newNode->next = head;
        return;
    }

    SingleList<T>* temp = head;
    while (temp->next != head) {
        temp = temp->next;
    }
    temp->next = newNode;
    newNode->next = head;
}
```

```cpp
void sllDelete(const T& findData, SingleList<T>*& head) {
    if (head == nullptr) return;

    SingleList<T>* currNode = head;
    SingleList<T>* prevNode = nullptr;


    do {
        if (currNode->data == findData) break;
        prevNode = currNode;
        currNode = currNode->next;
    } while (currNode != head);

    if (currNode->data != findData) return;


    if (currNode == head && currNode->next == head) {
        delete head;
        head = nullptr;
        return;
    }


    if (currNode == head) {
        SingleList<T>* last = head;
        while (last->next != head) {
            last = last->next;
        }
        head = head->next;
        last->next = head;
    } else {
        prevNode->next = currNode->next;
    }

    delete currNode;
}
```

```cpp
template <typename T>
void sllDeleteList(SingleList<T>*& head) {
    if (head == nullptr) return;

    SingleList<T>* currNode = head;
    SingleList<T>* nextNode = nullptr;

    do {
        nextNode = currNode->next;
        delete currNode;
        currNode = nextNode;
    } while (currNode != head);

    head = nullptr;
}

template <typename T>
SingleList<T>* getPreviousNode(SingleList<T>* head, SingleList<T>* current) {
    if (head == nullptr || current == nullptr) return nullptr;

    SingleList<T>* temp = head;
    while (temp->next != current) {
        temp = temp->next;
    }
    return temp;
}

#endif
```

Header File

```cpp
#include <iostream>
#include <string>
#include "singly_ll.h"
```

```cpp
void clearBuffer() {
    char c;
    while (std::cin.get(c) && c != '\n');
}

void playPlaylist(SingleList<std::string>* head) {
    if (head == nullptr) {
        std::cout << "\nPlaylist is empty! Add some songs first.\n";
        return;
    }

    SingleList<std::string>* currentSong = head;
    char playerChoice = ' ';

    while (true) {
        std::cout << "\n----------------------------------------\n";
        std::cout << "NOW PLAYING: >>> " << currentSong->data << " <<<\n";
        std::cout << "----------------------------------------\n";
        std::cout << "[N]ext Song | [P]revious Song | [S]top Playing\n";
        std::cout << "Choice: ";
        std::cin >> playerChoice;

        if (playerChoice == 'N' || playerChoice == 'n') {
            currentSong = currentSong->next;
        }
        else if (playerChoice == 'P' || playerChoice == 'p') {
            currentSong = getPreviousNode(head, currentSong);
        }
        else if (playerChoice == 'S' || playerChoice == 's') {
            std::cout << "Stopping player mode...\n";
            break;
        }
        else {
            std::cout << "Invalid player choice. Please use the valid options.\n";
        }
    }
}
```

```cpp
int main() {
    char choice = ' ';
    std::string newSong;

    SingleList<std::string>* head = nullptr;

    sllinsertEnd(std::string("MEOWMEOW"), head);
    sllinsertEnd(std::string("ANIMAL"), head);
    sllinsertEnd(std::string("HALA"), head);

    std::cout << "Welcome to Spotify!\n";
    std::cout << "==============================\n";
    std::cout << "Just created a playlist!\n";
    std::cout << "==============================\n\n";

    while (choice != 'D' && choice != 'd') {
        std::cout << "\n----- Current Playlist ----- \n";
        listTraversal(head);

        std::cout << "\nChoose action below:\n";
        std::cout << "A. Play playlist (Interactive)\n";
        std::cout << "B. Add a song to the playlist\n";
        std::cout << "C. Remove a song from the playlist\n";
        std::cout << "D. Exit\n";
        std::cout << "Enter your choice: ";
        std::cin >> choice;

        if (choice == 'A' || choice == 'a') {
            playPlaylist(head);
        }
        else if (choice == 'B' || choice == 'b') {
            clearBuffer();
            std::cout << "Enter the song name to add: ";
            std::getline(std::cin, newSong);
            sllinsertEnd(newSong, head);
            std::cout << "\"" << newSong << "\" added successfully.\n";
        }
```

```
        else if (choice == 'C' || choice == 'c') {
            clearBuffer();
            std::cout << "Enter the song name to remove: ";
            std::getline(std::cin, newSong);
            sllDelete(newSong, head);
            std::cout << "Removal requested for: \"" << newSong << "\".\n";
        }
        else if (choice != 'D' && choice != 'd') {
            std::cout << "Invalid choice. Please select A, B, C, or D.\n";
        }
    }

    sllDeleteList(head);
    std::cout << "\nExiting the program. Goodbye!\n";

    return 0;
}
```

Driver Function

OUTPUT

```
Welcome to Spotify!
==============================
Just created a playlist!
==============================


----- Current Playlist -----
1. MEOWMEOW
2. ANIMAL
3. HALA

Choose action below:
A. Play playlist (Interactive)
B. Add a song to the playlist
C. Remove a song from the playlist
D. Exit
Enter your choice: A
```

Menu

```
Enter your choice: A

----------------------------------------
NOW PLAYING: >>> MEOWMEOW <<<
----------------------------------------
[N]ext Song | [P]revious Song | [S]top Playing
Choice: n

----------------------------------------
NOW PLAYING: >>> ANIMAL <<<
----------------------------------------
[N]ext Song | [P]revious Song | [S]top Playing
Choice: p

----------------------------------------
NOW PLAYING: >>> MEOWMEOW <<<
----------------------------------------
[N]ext Song | [P]revious Song | [S]top Playing
Choice: S
Stopping player mode...
```

Option A: Play the playlist

```
----- Current Playlist -----
1. MEOWMEOW
2. ANIMAL
3. HALA

Choose action below:
A. Play playlist (Interactive)
B. Add a song to the playlist
C. Remove a song from the playlist
D. Exit
Enter your choice: B
Enter the song name to add: Hootie Fruitie
"Hootie Fruitie" added successfully.

----- Current Playlist -----
1. MEOWMEOW
2. ANIMAL
3. HALA
4. Hootie Fruitie

Choose action below:
A. Play playlist (Interactive)
B. Add a song to the playlist
C. Remove a song from the playlist
D. Exit
Enter your choice:
```

Option B: Adding a song to the playlist

```
----- Current Playlist -----
1. MEOWMEOW
2. ANIMAL
3. HALA
4. Hootie Fruitie

Choose action below:
A. Play playlist (Interactive)
B. Add a song to the playlist
C. Remove a song from the playlist
D. Exit
Enter your choice: C
Enter the song name to remove: MEOWMEOW
Removal requested for: "MEOWMEOW".

----- Current Playlist -----
1. ANIMAL
2. HALA
3. Hootie Fruitie

Choose action below:
A. Play playlist (Interactive)
B. Add a song to the playlist
C. Remove a song from the playlist
D. Exit
Enter your choice:
```

Option C: Removing a song from the Playlist

```
----- Current Playlist -----
1. ANIMAL
2. HALA
3. Hootie Fruitie

Choose action below:
A. Play playlist (Interactive)
B. Add a song to the playlist
C. Remove a song from the playlist
D. Exit
Enter your choice: D

Exiting the program. Goodbye!
```

Option D: Exiting the program

CODE ANALYSIS

In order to create a circular singly linked list a structural change must be done to the header file. Instead of pointing the last node to the nullptr, it is pointed to the head which results in a circular linked list that is appropriate to the usage of our code. The code initializes three songs by default which then asks for the user to perform 4 actions. Firstly the play playlist is interactive where it displays the current song playing which the user is allowed to either loop through the playlist by inputting the action that they want to do which are Previous, Skip or Stop the playing. The second option works by inserting the newData into the end of the linked list using the function sllinsertEnd. The third option allows the user to delete a specific song, this works by using the function sllDelete which traverses the linked list in order to find the song that the user wants to delete. Lastly the fourth option breaks the loop and ends the program.

CONCLUSION

Through this activity, I learned how linked lists actually work and how much flexibility pointers give us. By changing the last node to point back to the head instead of nullptr, I was able to turn a singly linked list into a circular one, which ended up being the what I need for the music playlist. Utilizing linked lists showed me how useful this structure is for real applications, especially when handling actions like looping through songs, adding new tracks to the end, or deleting specific ones on the fly. Overall, this lab gave me a much clearer, hands-on understanding of how to manage memory properly and alter structures to fit project needs.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

QUEZON CITY

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.